

Implementing Unsupervised Learning Algorithms from scratch

Objective

The goal is to implement the **K-Means clustering algorithm** (from scratch or minimally assisted), understand the concept of **cluster formation and centroid optimization**, and analyze the impact of hyperparameters like the number of clusters (K) and initialization strategy.

Dataset

Use the **Wine Dataset** from the UCI repository:

- Dataset name: Wine Dataset
- Link: <https://archive.ics.uci.edu/ml/datasets/wine>

Description:

- Multiclass dataset (3 wine classes, but labels will NOT be used for training)
- 13 numerical features
- Task: Discover natural groupings (clusters) in the data

Part (a): Implementation of the K-Means Model [50]

1. Understanding the K-Means Algorithm

K-Means is an **unsupervised learning algorithm** used to partition data into K clusters. It works by minimizing the within-cluster variance.

Steps:

1. Initialize Centroids:

Randomly select K data points as initial centroids.

2. Assign Clusters:

Assign each data point to the nearest centroid using Euclidean distance.

3. Update Centroids:

Compute new centroids as the mean of all points assigned to each cluster.

4. Check for Convergence:

Repeat steps 2–3 until centroids stop changing (or max iterations reached).

5. Final Clusters:

Output cluster assignments and centroid locations.

2. Dataset Preparation

- Use all features
- **Do NOT use class labels for training**
- Labels may be used **only for evaluation later**

3. Data Preprocessing

- Standardize features (mean = 0, variance = 1)
- Important because K -Means relies on distance calculations

4. Task: Implement the K-Means Model

Implement a class `KMeansClustering` with the following methods:

Required methods:

- `__init__(K, max_iters)` : To initialize the number of clusters and maximum iterations.
- `fit(X_train)` : To train the model on the training data.
- `predict(X_test)` : To assign cluster labels to new data points.

Implementation Guidelines

Use the standard K-Means objective:

Minimize intra-cluster variance:

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} ||x_i - \mu_k||^2$$

Where:

- μ_k = centroid of cluster k
- C_k = points in cluster k

Steps:

1. Random centroid initialization
2. Assign points to nearest centroid
3. Update centroids

4. Repeat until convergence

5. Training the Model

Split the preprocessed Wine dataset into a training set (80%) and a testing set (20%). Use the fit method to prepare your model.

Part (b): Evaluation and Visualization [50]

1. Hyperparameter Tuning

Evaluate performance for different values of:

- Number of clusters: $K = 2, 3, 4, 5, 6$
- Initialization strategies:
 - Random
 - K-Means++

Metrics to Report:

- Inertia (Within-cluster sum of squares)
- Silhouette Score

Present results in a table.

2. Cluster Evaluation (Using Ground Truth Labels)

Even though this is unsupervised:

- Compare clusters with actual labels using:
 - Adjusted Rand Index (ARI)
 - Accuracy via best label mapping

(You may use sklearn for evaluation only)

3. Visualization

Step 1: Reduce to 2 Dimensions

Use: Principal Component Analysis (PCA)

Step 2: Plot Clusters

Create plots for:

- K -Means clustering results
- True labels (for comparison)

Step 3: Decision Regions (Optional)

- Visualize Voronoi-like cluster boundaries in 2D space

4. Elbow Method Analysis

Plot:

- K vs Inertia

Explain:

- Optimal K selection
- Trade-off between complexity and fit

5. Analysis and Report

Provide a clear, well-structured report that explains your implementation, experimental findings, and conceptual understanding of the K-Means algorithm.

Implementation

- Explain how you implemented the **K-Means clustering algorithm** from scratch. Describe the overall workflow of your code, including how data flows through different steps of the algorithm.
- Clearly describe how centroids are initialized, how data points are assigned to clusters, and how centroids are updated in each iteration.
- Explain the **convergence criteria** used in your implementation (e.g., no change in centroids, minimal movement, or reaching a maximum number of iterations).

Challenges faced:

- Discuss the sensitivity of K -Means to **initial centroid selection** and how it affected your results.
- Explain how you handled or observed cases of **empty clusters** (i.e., when no points are assigned to a centroid).
- Describe any **convergence issues**, such as slow convergence or oscillations, and how you addressed them.

Hyperparameter Analysis

- Analyze the effect of different values of K (**number of clusters**) on the clustering results.
- Discuss the concepts of:
 - **Under-clustering** (too few clusters, leading to overly broad groupings)
 - **Over-clustering** (too many clusters, leading to fragmented groups)
- Compare different initialization strategies:
 - Random initialization
 - K -Means++ initialization
- Explain how initialization impacts convergence speed, stability, and final clustering quality.

Cluster Quality

- Explain how to interpret the **silhouette score**, including what high and low values indicate about cluster separation and cohesion.
- Discuss why **inertia (within-cluster sum of squares)** alone is not sufficient for evaluating clustering performance.
- Highlight the importance of using multiple evaluation metrics to get a more reliable assessment of clustering quality.

Conceptual Insight

- Explain the key concepts behind clustering, including:
 - **Cluster compactness** (how close points in the same cluster are)
 - **Cluster separation** (how distinct different clusters are from each other)
- Discuss why K-Means implicitly assumes **spherical clusters of similar size** due to its reliance on Euclidean distance.
- Reflect on how these assumptions affect performance on real-world datasets.

Limitations of K-Means

- Discuss the major limitations of the K-Means algorithm, including its sensitivity to:
 - Initial centroid placement
 - Presence of **outliers**, which can significantly distort cluster centers
 - **Non-spherical or unevenly sized clusters**, where K-Means may fail to capture true data structure
- Provide examples (based on your observations, if possible) where K-Means performs poorly.

Deliverables

Submit the following components as part of your assignment:

- Well-documented source code for your K-Means implementation
- A table summarizing results for different values of K and evaluation metrics
- Relevant plots, including:
 - Cluster visualizations (after dimensionality reduction, if applicable)
 - Elbow curve showing K vs inertia
- A final report (PDF format) summarizing your methodology, experiments, results, and insights

Bonus Challenge

1. K-Means vs Hierarchical Clustering

Compare your K-Means results with hierarchical clustering:

- Discuss differences in:
 - Interpretability of results (e.g., dendrogram vs flat clusters)
 - Performance on the dataset
 - Flexibility in capturing different cluster shapes

2. DBSCAN Comparison

Compare with:

- DBSCAN

Focus on:

- Ability to handle **noise and outliers**
- Performance on **arbitrarily shaped clusters**
- Situations where DBSCAN outperforms K-Means (you may use sklearn for DBSCAN implementation)

3. PCA + Clustering Pipeline

- Apply Principal Component Analysis (PCA) before clustering
- Compare clustering performance before and after dimensionality reduction
- Discuss whether PCA improves cluster separability and visualization

Important Note

- Do **NOT** use:
 - `sklearn.cluster.KMeans` for Part (a) (core implementation must be done from scratch)
- You **MAY** use sklearn for:
 - Evaluation metrics (e.g., silhouette score, ARI)
 - Dimensionality reduction (PCA)
 - Comparison models in Part (b)